HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO HÀNG TUẦN

Xây dựng Chatbot hỏi đáp về lịch sử Việt Nam sử dụng GraphRAG

Lớp: E22HTTT

Tên: Nguyễn Đức Anh

ID: B22DCDT017

Người hướng dẫn: Kim Ngọc Bách

Hà Nội – 2026

Weekly Report 10

I. OVERVIEW OF PROGRESS IN THE PAST WEEK

Over the past week, the team has resolved a major limitation of the Knowledge Graph: while the graph is very strong in logical connections and relationships, it lacks the detailed historical context and progression.

To overcome this, the team researched and successfully implemented a Hybrid RAG Architecture (Graph + Vector) on the same Neo4j database. Simultaneously, the team upgraded the Web interface, adding features specifically designed to support the project defense before the Evaluation Committee.

II. DETAILS OF COMPLETED TASKS

1. Building the Vector Database Module on Neo4j (Semantic Search)

In traditional retrieval systems (Keyword Search), computers only search based on exact keyword matches. This causes difficulties when users ask using synonyms or different phrasing. To resolve this, the team applied Semantic Search techniques via a Vector Database built directly into Neo4j. This process was executed through 3 core technical steps:

- **Text Embeddings:** Instead of storing raw text, the team utilized OpenAI's specialized text-embedding-3-small LLM model. This model acts to "read and understand" each Vietnamese historical text segment (chunks) and encodes its entire meaning into an array of real numbers consisting of 1536 dimensions (vector). Two paragraphs with similar historical meanings (even if using different vocabulary) will have their vector coordinates located close to each other in the multidimensional space.

```python
def get_embedding(text):
    """Hàm biến văn bản tiếng Việt thành mảng 1536 con số"""
    response = client.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    return response.data[0].embedding
```

- **Storing as Nodes in the Graph:** The original text segments (text) along with their corresponding vector arrays (embedding) are pushed to Neo4j via the Cypher MERGE command. They are stored as independent Nodes labeled :Chunk. This enables centralized storage for the system, eliminating the need for a third-party database (such as Pinecone or Milvus).
- **Setting up Vector Index and Cosine Similarity Algorithm:** To optimize search speeds across thousands of vectors, the team established a VECTOR INDEX (named chunk_vector) directly on Neo4j. The system is configured to use the Cosine Similarity algorithm. When a user asks a question, the question is also converted into a vector. The Cosine algorithm measures the angle between the question vector and the text vectors in the database; the smaller the angle (as cosine approaches 1), the closer the semantic meaning. As a result, the system can retrieve the historical text segments that best match the user's intent.

```python
session.run("""
    CREATE VECTOR INDEX chunk_vector IF NOT EXISTS
    FOR (c:Chunk) ON (c.embedding)
    OPTIONS {indexConfig: {
        `vector.dimensions`: 1536,
        `vector.similarity_function`: 'cosine'
    }}
""")
```

```
session.run("""
    MERGE (c:Chunk {id: $chunk_id})
    SET c.text = $text, c.embedding = $embedding
""", chunk_id=chunk_id, text=text, embedding=vector_data)
```

2. Developing the Hybrid RAG Engine

Standard RAG (Vector-only) often lacks strict logic, whereas GraphRAG lacks detailed event progression. Therefore, the team combined both methods to form a Hybrid RAG architecture. The system triggers a 3-step parallel processing flow:

- **Step 1: Vector Query (Extracting detailed context - Unstructured Data)**
    - The system uses the established Vector interpolation algorithm to scan the entire database.
    - Through LangChain's similarity_search(query, k=3) function, the system extracts the 3 text segments (chunks) with the highest semantic similarity to the question.
    - **Purpose:** To provide the AI with "context," "detailed progressions," and "specific dates" that are written in long-form text, which the Graph cannot fully represent.

```
vector_docs = vector_store.similarity_search(user_query, k=3)
vector_context = "\n\n".join([f"[Đoạn văn {i+1}]: {doc.page_content}" for i, doc in enumerate(vector_docs)])
```

Step 2: Graph Query (Extracting relational logic - Structured Data)

- The system uses GraphCypherQAChain to execute the Text-to-Cypher task. The LLM model (gpt-4o-mini) acts as a data engineer, translating Vietnamese questions into the Cypher query language.
- Thanks to strictly engineered Prompts, the AI consistently uses the MATCH (n:Entity)-[r]->(m:Entity) structure combined with a fuzzy search mechanism (toLower() and CONTAINS).

- The system executes the Cypher command on the Graph and returns a structured data array.
- **Purpose:** To provide the AI with absolute logical truths (e.g., Node "US" -> Relationship "IMPLEMENTS_PLOT" -> Node "Special War"), preventing the Chatbot from hallucinating or misrepresenting historical factions, figures, or actions.

```
try:
    graph_res = graph_chain.invoke({"query": user_query})
    graph_context = str(graph_res['result'])
except:
    graph_context = "Không có dữ liệu đồ thị cụ thể cho câu hỏi này."
```

Step 3: Multi-thread Synthesizer (The Synthesis LLM)

- Instead of answering directly from a single source, the system feeds both Vector Results and Graph Results into a massive System Prompt acting as a "knowledgeable Vietnamese History Professor".
- At this stage, the LLM strictly serves as a "synthesizing and drafting" tool. It is bound by the rule: It is only allowed to answer based on the two provided sources. It uses the logic from the Graph as the main outline and uses the sentences from the Vector results to flesh out and supplement the details.
- The final result is a coherent, historically accurate answer complete with event progression, presented scientifically (using bullet points) in accordance with the learning system's requirements.

```python
final_prompt = ChatPromptTemplate.from_messages([
    ("system", """Bạn là một Giáo sư Lịch sử Việt Nam uyên bác.
    Nhiệm vụ của bạn là trả lời câu hỏi dựa trên 2 nguồn tài liệu:
    - 'Dữ liệu Đồ thị': Cung cấp các mối quan hệ logic chính xác.
    - 'Dữ liệu Văn bản': Cung cấp diễn biến và bối cảnh chi tiết.

    YÊU CẦU TRẢ LỜI:
    - Nếu người dùng yêu cầu 'trình bày', 'chi tiết', 'kể về', bạn phải viết dài, đầy đủ diễn biến, không được tóm tắt sơ sài.
    - Sử dụng gạch đầu dòng để phân tách các ý chính.
    - Luôn ưu tiên trả lời dựa trên nội dung có trong tài liệu cung cấp. Nếu một nguồn bị thiếu, hãy dùng nguồn còn lại.
    - Giọng văn trang trọng, khách quan."""),
    ("user", """Câu hỏi của sinh viên: {question}

    --- TÀI LIỆU ĐỒ THỊ ---
    {graph_context}

    --- TÀI LIỆU VĂN BẢN VECTOR ---
    {vector_context}
    """)
])

# Sinh câu trả lời cuối cùng
response_chain = final_prompt | final_llm
full_response = response_chain.invoke({
    "question": user_query,
    "graph_context": graph_context,
    "vector_context": vector_context
}).content

st.write(full_response)
```